Universidad San Carlos de Guatemala

Facultad de Ingeniería

Ingeniería en Ciencias y Sistemas

Software Avanzado

Práctica 4

Actualización de Documentación y Testeo de Colas

Proyecto: DeliverEats - Fase 2

Carné: 201114493

Fecha de entrega: 25/02/2026

1. Actualización de Requerimientos Funcionales

Los siguientes requerimientos funcionales amplían y actualizan los definidos en la Práctica 1, incorporando los nuevos flujos asíncronos y la infraestructura Kubernetes de la Fase 2 de DeliverEats.

1.1 Requerimientos Existentes (Fase 1) — Confirmados

- RF-01: El sistema debe permitir el registro e inicio de sesión de usuarios con roles: CLIENTE, DRIVER y ADMIN.
- RF-02: El sistema debe permitir la gestión de restaurantes y sus menús (CRUD de ítems).
- RF-03: El sistema debe permitir a los clientes crear y consultar órdenes con estados: CREADA, EN_PROCESO, FINALIZADA, RECHAZADA.
- RF-04: El sistema debe permitir la gestión de entregas con estados: EN_CAMINO, ENTREGADO, CANCELADO.
- RF-05: El sistema debe generar facturas automáticamente al completarse una orden.
- RF-06: El sistema debe autenticar mediante tokens JWT con expiración configurable.

1.2 Nuevos Requerimientos Funcionales (Fase 2)

- RF-07: Comunicación Asíncrona vía Cola de Mensajes — El Order-Service debe publicar un mensaje en la cola (RabbitMQ) cada vez que se crea una nueva orden. El Restaurant-Service debe suscribirse y consumir dicho mensaje.
- RF-08: Orquestación con Kubernetes — Todos los microservicios deben desplegarse y escalar dentro de un clúster de Kubernetes usando Deployments y Services.
- RF-09: Pipeline CI/CD — El sistema debe contar con un pipeline automatizado (GitHub Actions) que construya imágenes Docker, ejecute pruebas y despliegue al clúster al hacer push a la rama principal.
- RF-10: Estrategia de Rollout y Rollback — El sistema debe soportar actualizaciones sin downtime mediante RollingUpdate y rollback automático ante fallos de health checks.
- RF-11: Flujo JWT Actualizado — El API Gateway debe validar el token JWT antes de redirigir cualquier petición a los microservicios internos; los servicios internos confían en el gateway.

2. Actualización de Requerimientos NO Funcionales

2.1 Requerimientos Existentes (Fase 1) — Actualizados

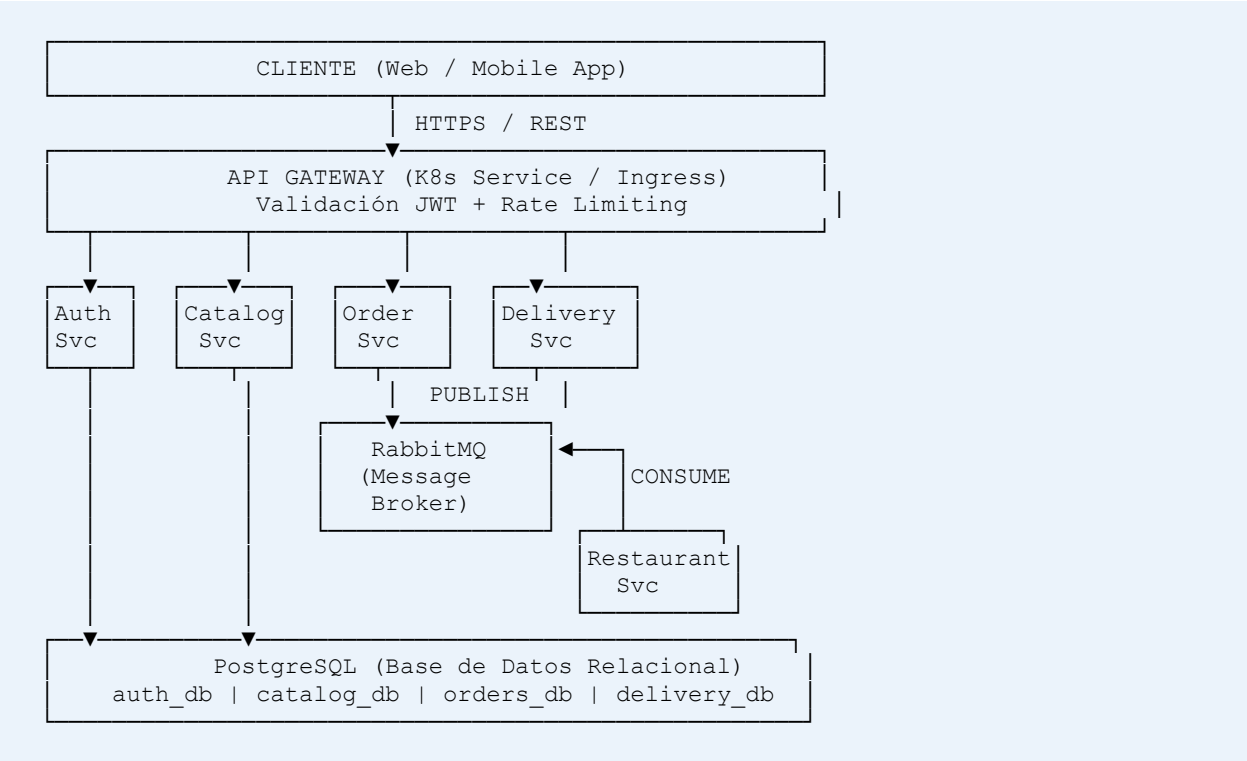
- RNF-01 Disponibilidad: El sistema debe garantizar una disponibilidad del 99.5%. Con Kubernetes se añade alta disponibilidad mediante réplicas mínimas de 2 pods por servicio crítico.
- RNF-02 Seguridad: Toda comunicación debe ir cifrada. Los secretos (contraseñas de BD, claves JWT, credenciales RabbitMQ) deben gestionarse como Kubernetes Secrets.
- RNF-03 Escalabilidad: El sistema debe escalar horizontalmente. Con K8s se define HorizontalPodAutoscaler (HPA) para escalar en base a CPU/memoria.
- RNF-04 Rendimiento: El tiempo de respuesta de las APIs REST no debe exceder 300ms en el percentil 95.

2.2 Nuevos Requerimientos NO Funcionales (Fase 2)

- RNF-05 Resiliencia del Broker: RabbitMQ debe desplegarse con persistencia de mensajes (durable queues) para evitar pérdida de mensajes ante reinicios del broker.
- RNF-06 Observabilidad: Los pods deben exponer health checks (/health) para liveness y readiness probes de Kubernetes.
- RNF-07 Portabilidad: Las imágenes Docker deben ser multi-stage para reducir tamaño y deben publicarse en un registry (Docker Hub o GitHub Container Registry).
- RNF-08 Trazabilidad del Pipeline: Cada ejecución del pipeline CI/CD debe generar logs auditables y notificaciones de fallo.
- RNF-09 Separación de Configuración: Toda configuración de entorno (URLs, puertos, nombres de colas) debe inyectarse vía ConfigMaps de Kubernetes, no hardcodeada en el código.

3. Actualización del Diagrama de Arquitectura de Alto Nivel

La arquitectura evoluciona de contenedores Docker simples (Fase 1) hacia un clúster Kubernetes con mensajería asíncrona (Fase 2). El siguiente diagrama textual describe los componentes y sus relaciones:



Cambios clave respecto a Fase 1

- Se añade RabbitMQ como broker de mensajería asíncrona entre Order-Service y Restaurant-Service.
- El API Gateway ahora valida JWT centralmente; los servicios internos ya no validan tokens individualmente.
- Se introduce el Restaurant-Service como microservicio independiente del Catalog-Service.
- Toda comunicación interna usa DNS de Kubernetes (cluster-internal).

4. Diagrama de Despliegue en Kubernetes

El diagrama de despliegue muestra cómo los componentes de DeliverEats se distribuyen dentro del clúster de Kubernetes, incluyendo Namespaces, Deployments, Services e Ingress.

```
CLÚSTER KUBERNETES (delivereats-cluster)

Namespace: delivereats-prod
┌──────────────────────────────────────────────────────────────────────┐
│ Ingress Controller (NGINX)                                           │
│   api.delivereats.com → api-gateway-service:80                       │
│                                                                      │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐   │
│  │ Deployment:      │  │ Deployment:      │  │ Deployment:      │   │
│  │ api-gateway      │  │ auth-service     │  │ catalog-service  │   │
│  │ replicas: 2      │  │ replicas: 2      │  │ replicas: 2      │   │
│  │ Port: 3000       │  │ Port: 3001       │  │ Port: 3002       │   │
│  └──────────────────┘  └──────────────────┘  └──────────────────┘   │
│                                                                      │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐   │
│  │ Deployment:      │  │ Deployment:      │  │ Deployment:      │   │
│  │ order-service    │  │ restaurant-svc   │  │ delivery-svc     │   │
│  │ replicas: 2      │  │ replicas: 2      │  │ replicas: 2      │   │
│  │ Port: 3003       │  │ Port: 3004       │  │ Port: 3005       │   │
│  └──────────────────┘  └──────────────────┘  └──────────────────┘   │
│          │ PUBLISH                ▲ CONSUME                          │
│          └──────────────────────┘                                    │
│                                                                      │
│  ┌──────────────────────────────┐  ┌──────────────────────────────┐ │
│  │ StatefulSet: RabbitMQ        │  │ StatefulSet: PostgreSQL      │ │
│  │ replicas: 1                  │  │ replicas: 1                  │ │
│  │ PVC: rabbitmq-data (5Gi)     │  │ PVC: pg-data (20Gi)          │ │
│  │ Port: 5672 (AMQP)            │  │ Port: 5432                   │ │
│  │      15672 (Management UI)   │  │                              │ │
│  └──────────────────────────────┘  └──────────────────────────────┘ │
└──────────────────────────────────────────────────────────────────────┘
```

Recursos de Kubernetes por componente

Componente	Tipo K8s	Réplicas	CPU Req.	Mem Req.
api-gateway	Deployment	2	100m	128Mi
auth-service	Deployment	2	100m	128Mi
catalog-service	Deployment	2	100m	128Mi
order-service	Deployment	2	150m	256Mi
restaurant-service	Deployment	2	150m	256Mi
delivery-service	Deployment	2	100m	128Mi
rabbitmq	StatefulSet	1	250m	512Mi
postgresql	StatefulSet	1	250m	512Mi

5. Actualización del Esquema de Base de Datos

El esquema original de la Práctica 1 se mantiene con las tablas: Usuario, Restaurante, ItemMenu, Orden, Entrega y Factura. Para la Fase 2 se agregan las siguientes tablas para soporte de mensajería asíncrona y auditoría:

5.1 Nuevas Tablas

Tabla: EventoMensaje (Auditoría de mensajes en cola)

Campo	Tipo	Restricción	Descripción
id_evento	INT	PK	Identificador único
tipo_evento	VARCHAR(50)	NOT NULL	Ej: ORDER_CREATED
id_orden	INT	FK → Orden	Referencia a la orden
payload	TEXT	NOT NULL	JSON del mensaje
estado	VARCHAR(20)	NOT NULL	PUBLICADO/CONSUMIDO
fecha_evento	DATETIME	NOT NULL	Timestamp del evento

Script SQL de actualización

```sql
-- ====================================
-- Actualización BD DeliverEats - Fase 2
-- ====================================

-- Tabla para auditoría de mensajes en cola
CREATE TABLE EventoMensaje (
    id_evento       INT           PRIMARY KEY AUTO_INCREMENT,
    tipo_evento     VARCHAR(50)   NOT NULL,
    id_orden        INT           NOT NULL,
    payload         TEXT          NOT NULL,
    estado          VARCHAR(20)   NOT NULL DEFAULT 'PUBLICADO',
    fecha_evento    DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_evento_orden
        FOREIGN KEY (id_orden) REFERENCES Orden(id_orden)
);

-- Agregar columna password a Usuario (requerida para Auth)
ALTER TABLE Usuario ADD COLUMN password_hash VARCHAR(255) NOT NULL DEFAULT '';

-- Agregar índices para mejor rendimiento
CREATE INDEX idx_orden_estado ON Orden(estado);
CREATE INDEX idx_entrega_estado ON Entrega(estado);
CREATE INDEX idx_evento_tipo ON EventoMensaje(tipo_evento);
```

6. Diagramas de Actividades / Secuencia Actualizados

6.1 Flujo Asíncrono: Creación de Orden con RabbitMQ

El siguiente diagrama de secuencia muestra el flujo actualizado al crear una orden, incorporando la mensajería asíncrona:

```
Cliente   API-GW    Order-Svc   RabbitMQ   Restaurant-Svc   BD
   |         |           |              |               |      |
   |—POST /orders——▶|           |              |               |      |
   |         |——Valida JWT—▶|              |               |      |
   |         |           |——INSERT Orden——————————————————▶|
   |         |           |◀————————————————————————200 OK——|
   |         |           |——PUBLISH {order_id, items}—▶|              |
   |         |◀——201 Created——|              |              |      |
   |◀——201 Created——|           |              |              |      |
   |         |           |              |——CONSUME msg——▶|      |
   |         |           |              |              |——Log en consola
   |         |           |              |◀——ACK————————|      |
```

6.2 Flujo de Actividades: Pipeline CI/CD

- Push a rama main en GitHub.
- GitHub Actions detecta el push y dispara el workflow.
- Se ejecutan los tests unitarios. Si fallan, el pipeline se detiene y notifica.
- Se construyen las imágenes Docker multi-stage de cada microservicio.
- Las imágenes se publican en Docker Hub / GHCR con tag del commit SHA.
- Se aplican los manifiestos Kubernetes (kubectl apply) al clúster.
- Kubernetes ejecuta el RollingUpdate. Si los readiness probes fallan, ejecuta rollback automático.
- Se envía notificación de éxito o fallo al equipo.

7. Guía de Despliegue en Kubernetes

7.1 Pre-requisitos

- kubectl instalado y configurado apuntando al clúster.
- Clúster Kubernetes disponible (local con Minikube/Kind o en la nube: GKE, EKS, AKS).
- Docker instalado para construir imágenes.
- Imágenes de los microservicios publicadas en registry.

7.2 Paso 1: Crear el Namespace

```
kubectl create namespace delivereats-prod
kubectl config set-context --current --namespace=delivereats-prod
```

7.3 Paso 2: Crear Secrets

```
kubectl create secret generic db-secret \
  --from-literal=POSTGRES_USER=delivereats \
  --from-literal=POSTGRES_PASSWORD=secretpassword \
  --from-literal=POSTGRES_DB=delivereats_db

kubectl create secret generic jwt-secret \
  --from-literal=JWT_SECRET=mi_clave_secreta_jwt_2026

kubectl create secret generic rabbitmq-secret \
  --from-literal=RABBITMQ_USER=guest \
  --from-literal=RABBITMQ_PASSWORD=guest
```

7.4 Paso 3: Desplegar RabbitMQ (StatefulSet)

```
# rabbitmq-statefulset.yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: rabbitmq
spec:
  replicas: 1
  selector:
    matchLabels:
      app: rabbitmq
  template:
    spec:
      containers:
      - name: rabbitmq
        image: rabbitmq:3.12-management
        ports:
```

```
        - containerPort: 5672
        - containerPort: 15672
        envFrom:
        - secretRef:
            name: rabbitmq-secret
        resources:
          requests: {cpu: 250m, memory: 512Mi}
---
kubectl apply -f rabbitmq-statefulset.yaml
kubectl wait --for=condition=ready pod/rabbitmq-0 --timeout=120s
```

7.5 Paso 4: Desplegar Microservicios

```
# Desplegar cada microservicio:
kubectl apply -f k8s/order-service-deployment.yaml
kubectl apply -f k8s/restaurant-service-deployment.yaml

# Verificar despliegue:
kubectl get pods -n delivereats-prod
kubectl get services -n delivereats-prod
```

7.6 Estrategias de Rollout y Rollback

La estrategia de actualización de todos los Deployments es RollingUpdate con los siguientes parámetros:

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1        # 1 pod extra durante update
    maxUnavailable: 0  # 0 pods fuera en todo momento

# Rollback manual (cuando sea necesario):
kubectl rollout undo deployment/order-service

# Ver historial de rollouts:
kubectl rollout history deployment/order-service

# Rollback a una versión específica:
kubectl rollout undo deployment/order-service --to-revision=2
```

8. Pipeline CI/CD y Flujo JWT

8.1 Descripción del Pipeline CI/CD

El pipeline se implementa con GitHub Actions. El archivo .github/workflows/deploy.yml define el flujo completo:

```yaml
name: DeliverEats CI/CD Pipeline

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Run Tests
        run: npm test

  build-and-push:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - name: Login to Docker Hub
        uses: docker/login-action@v2
        with:
          username: ${{ secrets.DOCKER_USERNAME }}
          password: ${{ secrets.DOCKER_PASSWORD }}
      - name: Build & Push order-service
        run: |
          docker build -t user/order-service:${{ github.sha }} ./order-service
          docker push user/order-service:${{ github.sha }}

  deploy:
    needs: build-and-push
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to K8s
        run: |
          kubectl set image deployment/order-service \
            order-service=user/order-service:${{ github.sha }}
```

8.2 Variables y Secrets del Pipeline

Variable/Secret	Valor de Ejemplo	Descripción
DOCKER_USERNAME	mi_usuario_docker	Usuario de Docker Hub
DOCKER_PASSWORD	**** (token)	Token de acceso Docker Hub

KUBECONFIG	Base64 del kubeconfig	Config acceso al clúster K8s
POSTGRES_PASSWORD	**** (strong pass)	Contraseña de PostgreSQL
JWT_SECRET	**** (256-bit key)	Clave firma de tokens JWT
RABBITMQ_PASSWORD	**** (strong pass)	Contraseña de RabbitMQ
RABBITMQ_URL	amqp://guest:****@rabbitmq:5672	URL de conexión al broker

8.3 Descripción del Flujo JWT

El flujo de autenticación con JWT en la Fase 2 opera de la siguiente manera:

- 1. El cliente envía credenciales (email/password) al Auth-Service vía POST /auth/login.
- 2. El Auth-Service valida las credenciales contra la BD, y si son correctas genera un JWT firmado con HS256 y expira en 24 horas.
- 3. El token contiene el payload: { sub: id_usuario, rol: 'CLIENTE', iat: timestamp, exp: timestamp }.
- 4. En cada petición subsiguiente el cliente incluye el token en el header: Authorization: Bearer <token>.
- 5. El API Gateway intercepta la petición, extrae el token y lo verifica usando la clave secreta JWT_SECRET.
- 6. Si el token es válido, el gateway reenvía la petición al microservicio correspondiente, añadiendo headers internos X-User-Id y X-User-Role.
- 7. Los microservicios internos confían en esos headers sin re-validar el token.

Ejemplo de petición autenticada:

```
# Login:
POST /auth/login
Content-Type: application/json
{"email": "carlos@mail.com", "password": "mi_password"}

# Response:
{"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."}

# Usar token en petición protegida:
POST /orders
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
{"id_restaurante": 1, "items": [{"id_item": 1, "cantidad": 2}]}
```

9. Prueba de Concepto (PoC) RabbitMQ

Se implementa un canal de comunicación asíncrona básico entre el Order-Service (Productor) y el Restaurant-Service (Consumidor) usando RabbitMQ con la cola 'order_queue'.

9.1 docker-compose.yml (entorno local de prueba)

```yaml
version: '3.8'
services:
  rabbitmq:
    image: rabbitmq:3.12-management
    ports:
      - '5672:5672'
      - '15672:15672'
    environment:
      RABBITMQ_DEFAULT_USER: guest
      RABBITMQ_DEFAULT_PASS: guest
    healthcheck:
      test: rabbitmq-diagnostics ping
      interval: 10s
      timeout: 5s
      retries: 5
  order-service:
    build: ./order-service
    environment:
      RABBITMQ_URL: amqp://guest:guest@rabbitmq:5672
    depends_on:
      rabbitmq:
        condition: service_healthy
  restaurant-service:
    build: ./restaurant-service
    environment:
      RABBITMQ_URL: amqp://guest:guest@rabbitmq:5672
    depends_on:
      rabbitmq:
        condition: service_healthy
```

9.2 Order-Service — Productor (Node.js)

Archivo: order-service/index.js

```javascript
const amqp = require('amqplib');
const express = require('express');

const app = express();
app.use(express.json());

const RABBITMQ_URL = process.env.RABBITMQ_URL ||
'amqp://guest:guest@localhost:5672';
const QUEUE_NAME = 'order_queue';
```

```
let channel;

async function connectRabbitMQ() {
  try {
    const connection = await amqp.connect(RABBITMQ_URL);
    channel = await connection.createChannel();
    // Cola durable = persiste ante reinicios del broker
    await channel.assertQueue(QUEUE_NAME, { durable: true });
    console.log('[Order-Service] Conectado a RabbitMQ. Cola lista:', QUEUE_NAME);
  } catch (error) {
    console.error('[Order-Service] Error conectando a RabbitMQ:', error.message);
    setTimeout(connectRabbitMQ, 5000); // Retry en 5 segundos
  }
}

// Endpoint POST /orders — crea una orden y publica mensaje
app.post('/orders', async (req, res) => {
  const { id_restaurante, id_cliente, items } = req.body;

  // Simular creación de orden en BD
  const nuevaOrden = {
    id_orden: Math.floor(Math.random() * 10000),
    id_restaurante,
    id_cliente,
    items,
    estado: 'CREADA',
    fecha_orden: new Date().toISOString()
  };

  if (!channel) {
    return res.status(503).json({ error: 'Broker no disponible' });
  }

  // Publicar mensaje en la cola
  const mensaje = JSON.stringify({
    evento: 'ORDER_CREATED',
    orden: nuevaOrden
  });

  channel.sendToQueue(
    QUEUE_NAME,
    Buffer.from(mensaje),
    { persistent: true } // Mensaje persistente
  );

  console.log('[Order-Service] Mensaje publicado en cola:', QUEUE_NAME);
  console.log('[Order-Service] Payload:', mensaje);

  return res.status(201).json({ message: 'Orden creada', orden: nuevaOrden });
});

// Health check para K8s
app.get('/health', (req, res) => res.json({ status: 'ok' }));

const PORT = process.env.PORT || 3003;
app.listen(PORT, async () => {
  await connectRabbitMQ();
  console.log('[Order-Service] Corriendo en puerto', PORT);
});
```

Archivo: order-service/package.json

```
{
  "name": "order-service",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": { "start": "node index.js" },
  "dependencies": {
    "amqplib": "^0.10.3",
    "express": "^4.18.2"
  }
}
```

Archivo: order-service/Dockerfile

```
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY . .
EXPOSE 3003
CMD ["node", "index.js"]
```

9.3 Restaurant-Service — Consumidor (Node.js)

Archivo: restaurant-service/index.js

```
const amqp = require('amqplib');
const express = require('express');

const app = express();

const RABBITMQ_URL = process.env.RABBITMQ_URL ||
'amqp://guest:guest@localhost:5672';
const QUEUE_NAME = 'order_queue';

async function startConsumer() {
  try {
    const connection = await amqp.connect(RABBITMQ_URL);
    const channel = await connection.createChannel();

    // Declarar la misma cola que el productor
    await channel.assertQueue(QUEUE_NAME, { durable: true });

    // Procesar 1 mensaje a la vez (fair dispatch)
    channel.prefetch(1);

    console.log('[Restaurant-Service] Esperando mensajes en cola:', QUEUE_NAME);
```

```
    console.log('[Restaurant-Service] Presiona CTRL+C para salir...');

    channel.consume(QUEUE_NAME, (msg) => {
      if (msg !== null) {
        const contenido = JSON.parse(msg.content.toString());

        // Registrar el mensaje consumido en consola
        console.log('========================================');
        console.log('[Restaurant-Service] Mensaje recibido de la cola!');
        console.log('[Restaurant-Service] Evento:', contenido.evento);
        console.log('[Restaurant-Service] Orden ID:', contenido.orden.id_orden);
        console.log('[Restaurant-Service] Restaurante:',
contenido.orden.id_restaurante);
        console.log('[Restaurant-Service] Items:',
JSON.stringify(contenido.orden.items));
        console.log('[Restaurant-Service] Estado:', contenido.orden.estado);
        console.log('[Restaurant-Service] Fecha:', contenido.orden.fecha_orden);
        console.log('========================================');

        // Confirmar que el mensaje fue procesado (ACK)
        channel.ack(msg);
        console.log('[Restaurant-Service] ACK enviado - mensaje procesado
correctamente.');
      }
    });
  } catch (error) {
    console.error('[Restaurant-Service] Error:', error.message);
    setTimeout(startConsumer, 5000); // Retry en 5 segundos
  }
}

// Health check para K8s
app.get('/health', (req, res) => res.json({ status: 'ok' }));

const PORT = process.env.PORT || 3004;
app.listen(PORT, async () => {
  console.log('[Restaurant-Service] Corriendo en puerto', PORT);
  await startConsumer();
});
```

9.4 Verificación del PoC — Salida Esperada en Consola

Al ejecutar 'docker-compose up' y hacer una petición POST /orders, se espera la siguiente salida:

```
# Terminal Order-Service:
[Order-Service] Conectado a RabbitMQ. Cola lista: order_queue
[Order-Service] Corriendo en puerto 3003
[Order-Service] Mensaje publicado en cola: order_queue
[Order-Service] Payload: {"evento":"ORDER_CREATED","orden":{...}}

# Terminal Restaurant-Service:
[Restaurant-Service] Corriendo en puerto 3004
[Restaurant-Service] Esperando mensajes en cola: order_queue
========================================
[Restaurant-Service] Mensaje recibido de la cola!
[Restaurant-Service] Evento: ORDER_CREATED
```

```
[Restaurant-Service] Orden ID: 4821
[Restaurant-Service] Restaurante: 1
[Restaurant-Service] Items: [{"id_item":1,"cantidad":2}]
[Restaurant-Service] Estado: CREADA
[Restaurant-Service] Fecha: 2026-02-25T05:00:00.000Z
========================================
[Restaurant-Service] ACK enviado - mensaje procesado correctamente.
```

10. Instrucciones de Entrega

Estructura del repositorio

```
SA_PRACTICAS_<CARNET>/
├── PRACTICA1/                # (trabajo existente)
└── PRACTICA4/
    ├── documentacion/
    │   └── Practica4_<CARNET>.docx
    ├── order-service/
    │   ├── index.js
    │   ├── package.json
    │   └── Dockerfile
    ├── restaurant-service/
    │   ├── index.js
    │   ├── package.json
    │   └── Dockerfile
    ├── k8s/                  # Manifiestos K8s
    │   ├── rabbitmq-statefulset.yaml
    │   ├── order-service-deployment.yaml
    │   └── restaurant-service-deployment.yaml
    └── docker-compose.yml
```

Comandos para crear el tag

```
git add .
git commit -m "feat: Practica 4 - Documentacion Fase 2 y PoC RabbitMQ"
git tag v1.1.0
git push origin main --tags
```